

File Permissions in Linux

Project Description

I am a security professional at a large organization. I primarily work with their research team. Part of my job is to ensure that the users on this team have the appropriate permissions. This helps keep the system secure.

My job is to examine the existing permissions on the file system. I will need to determine if the permissions match the authorization that should be granted. If they do not match, I will need to change the permissions to authorize the appropriate users and remove any unauthorized access.

Check file and directory details

In order to ensure that the research team members have been granted sufficient permissions, that they are neither more nor less than necessary, I need to review these permissions first.

Using the **ls -la** command, I view the list of files in the projects directory and the permission system set for files in this directory.

```
researcher2@ffff46d5be53:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 18 09:59 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 18 10:42 ..
-rw--w---- 1 researcher2 research_team  46 Aug 18 09:59 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 18 09:59 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Aug 18 09:59 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Aug 18 09:59 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 18 09:59 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 18 09:59 project_t.txt
```

Describe the permissions string

To explain the character mask of permissions, I will take a file called **project_k.txt** as an example. First, it's worth explaining how to interpret the file permissions character mask. The 10-character string can be deconstructed to determine specific permissions:

- **1st character:** Indicates the file type (hyphen - for a regular file, **d** for a directory).
- **2nd-4th characters:** Indicate the read (**r**), write (**w**), and execute (**x**) permissions for the **user** (owner).
- **5th-7th characters:** Indicate the read (**r**), write (**w**), and execute (**x**) permissions for the **group**.
- **8th-10th characters:** Indicate the read (**r**), write (**w**), and execute (**x**) permissions for **others**.

We see that the first character is a hyphen because the file is not a directory. Using the example of the selected file, we see that the same rights are set for all three categories of users mentioned above,

namely for reading and writing the file. After the character mask of permissions for the three user groups, we see the name of the account that owns this file as well as the group it belongs to.

Change file permissions

The organization does not allow others to have write access to any files. Based on this logic, the file we chose for example has excessive permissions for the other user category. That's why as a cybersecurity specialist I restrict write permissions to a specific group on a file.

To do this, I use the **chmod** command, which changes the permissions on files and directories. The first argument indicates what permissions should be changed, and the second argument specifies the file or directory in question:

```
researcher2@ffff46d5be53:~/projects$ chmod o-w project_k.txt
```

After that, I need to make sure that the restrictions I imposed worked.

```
researcher2@ffff46d5be53:~/projects$ ls -la
...
-rw-rw-r-- 1 researcher2 research_team 46 Aug 18 09:59 project_k.txt
...
```

Now no users from the others group have permission to write to any of the files.

Change file permissions on a hidden file

The research team has archived **.project_x.txt**, so it is a hidden file. This file should not have write permissions for anyone, but the user and group should be able to read the file. We check the character mask of permissions again, as we can see, both the file owner and the members of his group have excessive permissions. It was also found that members of the file owner's group do not have permission to read the file.

To eliminate the violation and align with campaign policy, I change access rights of specific groups to "read only" using a combined command:

```
researcher2@ffff46d5be53:~/projects$ chmod u-w,g-w,g+r .project_x.txt
```

Then I check the permissions set and make sure everything works.

```
researcher2@ffff46d5be53:~/projects$ ls -la
...
-r--r----- 1 researcher2 research_team 46 Aug 18 09:59 .project_x.txt
...
```

Everything is correct.

Change directory permissions

The files and directories in the projects directory are owned by the user researcher2. Only researcher2 should have access to the drafts directory and its contents. During the analysis of the set permissions, it was discovered that the file owner group has excessive rights, namely the right to execute.

I change the permissions by prohibiting the directory owner's group from executing. After which I check the set permissions and make sure that everything is working correctly:

```
researcher2@ffff46d5be53:~/projects$ chmod g-x drafts
researcher2@ffff46d5be53:~/projects$ ls -la
...
drwx----- 2 researcher2 research_team 4096 Aug 18 09:59 drafts
...
```

Fine!

Summary

So, as a cybersecurity specialist, I conducted an audit of file access in the directory "projects" for all user groups. As a result of the audit of the access rights, violations were identified, namely insufficient powers or, conversely, excessive powers, which contradicts the best policies called "least privilege". As a result, permissions were changed to align with best practices and company policy.